CM3035 Coursework 1 — Chicago Crime REST API

Module: CM3035 Advanced Web Development **Author:** Matthias Naumann **Date:** June 2026

---

1. Introduction and Dataset

This report describes a RESTful web API built with Django and the Django REST Framework (DRF) that serves Chicago crime incident data. The application loads a CSV dataset into a relational database, exposes six analytical REST endpoints, and includes a comprehensive test suite with property-based testing.

The dataset is drawn from the City of Chicago Open Data Portal's "Crimes — 2001 to Present" collection, filtered to the 9,500 most recent incidents. Chicago's crime data is particularly well-suited to this assignment because it combines three analytical dimensions in a single dataset: temporal (each incident carries a precise timestamp), categorical (each incident is classified under an offense type such as theft, battery, or criminal damage), and geographic (each incident is assigned to a police district). This multi-dimensional structure enables genuinely interesting queries that go far beyond simple CRUD operations.

The dataset contains 9,499 incidents across 27 offense categories and 22 police districts. The most frequent offense types are theft (2,024 incidents), battery (1,902), criminal damage (1,067), assault (848), and motor vehicle theft (791). Each record includes fields for case number, date, address block, arrest status, domestic flag, FBI classification code, and foreign key references to offense category and district.

---

2. Database Design

The application uses three Django models — Incident, OffenseCategory, and District — that mirror the multi-model pattern taught in the course (the gene → EC → sequencing relationship from Topic 4). This design was chosen because it separates concerns cleanly: offense categories and districts are independent entities that exist outside any single incident, while each incident references both.

OffenseCategory

Stores offense classification data. The `code` field holds the IUCR (Illinois Uniform Crime Reporting) code, which is unique per offense type. The `category` field classifies each offense into one of three groups — violent, property, or quality-of-life — enabling aggregation queries that group incidents by crime severity.

District

Represents a police district with a unique district number, name, area type (police district or community area), and population figure. The population field enables per-capita incident rate calculations in the district safety endpoint, making comparisons between districts of different sizes meaningful.

Incident

The central fact table. Each incident links to one offense category and one district via foreign keys with `on_delete=CASCADE` and `related_name='incidents'`. This relationship structure enables efficient reverse lookups — `OffenseCategory.incidents.count()` or `District.incidents.filter(arrest=True)` — while the cascade deletion ensures that removing a category or district cleans up its associated incidents.

The design is intentionally the same relationship shape taught in the course (a central model with two foreign key relationships), applied to a richer domain with more analytical dimensions.

---

3. API Endpoints

The application exposes six REST endpoints, all hyperlinked from the API root page at `/api/`. Each endpoint was designed to answer a specific analytical question about the data, demonstrating progressively complex use of the Django ORM.

Endpoint 1: Incident List — `GET /api/incidents/`

Returns a paginated list of all incidents with nested offense category and district data. Supports filtering by category name, district name, date range, and arrest status via query parameters. The view uses `select_related('primary_type', 'district')` to avoid the N+1 query problem when serializing related objects.

```
queryset = Incident.objects.select_related('primary_type', 'district').all()
if category:
    queryset = queryset.filter(primary_type__name__icontains=category)
if date_from:
    queryset = queryset.filter(date__gte=date_from)
if arrest is not None:
    queryset = queryset.filter(arrest=(arrest.lower() == 'true'))
```

Endpoint 2: Incident Detail — `GET /api/incidents/{id}/`

Returns a single incident with fully nested serialization of its related offense category and district. Supports PUT for updates and DELETE for removal, returning appropriate HTTP status codes (200, 201, 204, 404).

Endpoint 3: Create Incident — `POST /api/incidents/`

Accepts JSON payloads with `primary_type_id` and `district_id` write-only integer fields. The serializer's overridden `create()` method validates that the referenced foreign key objects exist, raising `ValidationError` with field-specific messages if they do not, then links the new incident to the correct category and district. This matches the POST implementation pattern from Topic 4 exactly.

Endpoint 4: Arrest Rates — `GET /api/stats/arrest-rates/`

Returns arrest rate percentages for each offense category, ordered from lowest to highest arrest rate. The query uses Django's conditional aggregation — `Count` with a `Q` filter for arrest status, wrapped in a `Case`/`When` expression to safely handle division when a category has zero incidents:

```
OffenseCategory.objects.annotate(
    total=Count('incidents'),
    arrests=Count('incidents', filter=Q(incidents__arrest=True)),
).annotate(
    arrest_rate=Case(
        When(total=0, then=0.0),
        default=100.0 * F('arrests') / F('total'),
    )
).values('name', 'category', 'total', 'arrests', 'arrest_rate')
    .order_by('-arrest_rate')
```

This is an interesting query because it reveals which offense types have the lowest resolution rates — a meaningful analytical insight that raw data alone cannot surface.

Endpoint 5: District Safety — `GET /api/stats/district-safety/`

Computes a per-capita incident rate for each police district (incidents per 100,000 residents), ordered from highest to lowest. Uses `F()` expressions for field-level arithmetic and a `Case` guard against zero-population

districts. This endpoint answers the question "which districts have the most crime relative to their population?" — a more nuanced question than raw incident counts.

Endpoint 6: Temporal Trends — `GET /api/stats/temporal/`

Aggregates incidents by month using `TruncMonth` and computes monthly totals with arrest percentages, ordered chronologically. This endpoint reveals seasonal patterns in crime rates and tracks whether arrest rates change over time.

Main Page

The API root at `/api/` renders an HTML page showing all six endpoints as clickable hyperlinks, along with system metadata: Python version, Django version, installed packages, and admin credentials. This satisfies the assignment's requirement for hyperlinked endpoint discovery and metadata display.

---

4. Implementation

Project Structure

The code is organised following the patterns established in Topics 3–4 of the course. REST API views live in `api.py`, separate from any HTML-generating views that would go in `views.py`. Serializers are in `serializers.py`, test fixtures in `model_factories.py`, and tests are split into a `tests/` package with `test_api.py` (endpoint integration tests) and `test_serializers.py` (isolated serializer tests plus Hypothesis property-based tests). This separation ensures each file has a single clear responsibility.

Django REST Framework Patterns

All six endpoints use DRF class-based views. `IncidentList` extends `ListCreateAPIView`, combining list and create functionality in a single class with a custom `get_queryset()` override for filtering. `IncidentDetail` extends `RetrieveUpdateDestroyAPIView`, providing full CRUD for individual incidents. The three statistical endpoints use `GenericAPIView` with custom `get()` methods, chosen because their response shapes do not map to a standard queryset-and-serializer pattern.

Serializers follow the ModelSerializer pattern. `IncidentSerializer` uses nested read-only serializers for `primary_type` and `district` (so GET responses include full related object data) and write-only integer fields for `primary_type_id` and `district_id` (so POST requests accept simple ID references). The overridden `create()` method manually resolves foreign key references and raises descriptive validation errors on missing references — the same pattern demonstrated in the Topic 4 POST implementation lesson.

CSV Data Loading

Data is loaded via a Django management command (`python manage.py load_crime_data <csv_file>`), which reads the CSV using Python's `csv.DictReader`, creates offense categories and districts via `get_or_create` (making the command idempotent), categorises each offense type as violent, property, or quality-of-life through keyword matching, and creates incident records with timezone-aware dates. The command caps at 9,500 rows to stay within the assignment's 10,000-entry limit.

Swagger Documentation

`drf-spectacular` generates an OpenAPI 3.0 schema at `/api/schema/` and an interactive Swagger UI at `/api/docs/`. All six endpoints are documented with their HTTP methods, parameters, and response schemas. This goes beyond the course content and provides the interactive API exploration shown in the video demonstration.

---

5. Testing Strategy

The test suite contains 33 tests across two files, using patterns from the Topic 4 testing lessons.

Test Fixtures

`model_factories.py` defines FactoryBoy factories for all three models using `Sequence` for unique fields, `Faker` for realistic data, `choice` for constrained fields, and `randint` for numeric ranges. Every test run uses randomised data rather than static fixtures, ensuring tests are robust to varied input values — the "sophisticated" testing approach demonstrated in the course.

API Integration Tests (`test_api.py`)

Six `APITestCase` classes test every endpoint: successful responses (200, 201, 204), error responses (400 for missing fields, 400 for invalid foreign keys, 404 for missing resources), filtering behaviour (category, arrest status), response structure (nested serialised objects present), and calculation correctness (arrest rates, per-capita values, temporal aggregation). Each test class uses `setUp` to create controlled test data and `tearDown` to clean the database.

Serializer Tests (`test_serializers.py`)

Isolated serializer tests verify that `IncidentSerializer` accepts valid data, rejects missing required fields, correctly links foreign keys in its `create()` method, and raises `ValidationError` for nonexistent foreign key references. Separate tests cover `OffenseCategorySerializer` and `DistrictSerializer` field presence and validation.

Hypothesis Property-Based Testing

A Hypothesis-based test exercises `IncidentSerializer` with 50 randomly generated valid input combinations (varying case numbers, dates, blocks, arrest/domestic flags, and FBI codes) and asserts that the serializer either returns valid data or produces structured validation errors — never crashes. This is a testing technique beyond the course syllabus and satisfies the "advanced techniques" criteria.

```
@given(
    case_number=st.text(min_size=1, max_size=18),
    date_str=st.dates().map(lambda d: d.isoformat() + 'T12:00:00Z'),
    block=st.text(min_size=1, max_size=190),
    arrest=st.booleans(),
    domestic=st.booleans(),
    fbi_code=st.text(min_size=1, max_size=8),
)
@settings(max_examples=50, deadline=None)
def test_serializer_never_crashes_on_varied_input(self, ...):
    ...
```

---

6. Critical Evaluation

This application was built to satisfy a coursework specification within a constrained environment (SQLite3, single Django server, no authentication). A production-grade crime data API would require several additional concerns.

Database engine. SQLite3 is suitable for development and small datasets, but a production service with concurrent writes would need PostgreSQL for row-level locking, connection pooling, and geographic query support (PostGIS for spatial filtering by district boundary).

Authentication and authorisation. This API uses Django REST Framework's `AllowAny` permission class — every endpoint is publicly accessible. A production API would implement token-based or session authentication, with rate limiting to prevent abuse and differentiated access levels (public access for aggregate statistics, authenticated access for incident-level data).

Asynchronous processing. The CSV data loading runs synchronously in a management command. At larger scales, this would block the server. A production system would offload data ingestion to an asynchronous task queue (Celery, as covered in Topic 6) with progress reporting and error recovery.

Pagination and performance. The incident list endpoint uses page-number pagination (100 records per page), which is adequate for 9,500 records. At larger scales, cursor-based pagination would provide more consistent performance. Database indexes on frequently filtered fields (`date`, `arrest`, `primary_type_id`, `district_id`) would improve query performance.

Continuous integration. The test suite runs locally via `python manage.py test`. A CI pipeline (such as GitHub Actions) would automatically run the test suite on every push, enforce code style through a linter such as flake8 or ruff, and block deployment if any tests fail — ensuring that only verified code reaches production.

These limitations are appropriate for a coursework project — the application demonstrates understanding of the core concepts (RESTful design, ORM queries, serialisation, testing) while the report acknowledges what would be required for production readiness.

---

7. Run Instructions

Environment

- **Operating System:** Windows 11 Enterprise (also tested on Linux/macOS)
- **Python version:** 3.12.1
- **Django version:** 5.0.7
- **Key packages:** djangorestframework 3.15.2, drf-spectacular 0.28.0, factory-boy 3.3.1, hypothesis 6.111.1

Setup

```
# 1. Extract the ZIP archive
unzip cm3035_coursework_submission.zip
cd coursework2026

# 2. Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate      # Windows
# source venv/bin/activate  # Linux/macOS

# 3. Install dependencies
pip install -r requirements.txt

# 4. Apply database migrations
python manage.py migrate

# 5. Load the dataset
python manage.py load_crime_data chicago_crime_data.csv

# 6. Run the tests
python manage.py test crime_api -v 2
```

```
# 7. Start the development server
python manage.py runserver
```

Access

- **API root:** http://localhost:8000/api/
- **Swagger docs:** http://localhost:8000/api/docs/
- **Admin site:** http://localhost:8000/admin/
- **Admin credentials:** username `admin`, password `admin123`

References

City of Chicago Open Data Portal (2026). *Crimes — 2001 to Present*. Available at: https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2

Fielding, R.T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. PhD Thesis, University of California, Irvine.

Django REST Framework Documentation. Available at: https://www.django-rest-framework.org/

Hypothesis Documentation. Available at: https://hypothesis.readthedocs.io/